

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (Bsc.IT Semester III)
Data Structures Practical

Practical - IV

Roll No: S011	Name: Nidhi
Class : SYBSCIT	Batch : 1
Date of Assignment : 15-07-26	Date/Time of Submission : 15-07-26

Queue Application:

- a. Write a program to: a. Implement a queue using an array.

Code:

```
arr = []

def enqueue(data):
    arr.append(data)

def dequeue(data):
    arr.pop(data)

def show():
    print(arr)

enqueue(11)
enqueue(12)
enqueue(13)
enqueue(16)
show()
```

Output:

```
.py
[11, 12, 13, 16]
```

- b. Simulate a simple queuing system (e.g., customer service queue).

Code:

```
import time
que=[("Cus1", 5), ("Cus2", 2), ("Cus3", 7)]

while que:
    job, timer= que.pop()
    print(f"---Printing---: {job} {timer}")
    time.sleep(timer)
```

Output:

```
= RESTART: C:/Users/nidhi/Downloads  
.py  
----Printing---: Cus3 7  
----Printing---: Cus2 2  
----Printing---: Cus1 5
```

Curiosity Questions:

1. Where do you see queues in daily life?
2. Why is **FIFO (First In First Out)** fair in real life? Can you think of situations where it's *not* fair?
3. What will happen if you try to **enqueue more elements than the queue size**?
4. What if you **dequeue from an empty queue**?

5. Imagine you're designing a **hospital emergency system**. Should it use a simple queue or something else? (hint: priority queue).
6. If **5 customers arrive at the same time**, but the service desk can serve only 2 at once, how would you design that?